

# GitHub Customer Churn Predictor - Report

Final project report for a Dockerized FastAPI machine learning application using public GitHub activity data.

## 1. Project Overview

This project implements a containerized machine learning API to predict customer churn based on public GitHub user activity. The application collects user profile data from the GitHub REST API, generates behavioral and profile-based features, applies multiple feature selection methods, trains a classification model, and exposes the prediction model through a FastAPI service.

The application is packaged with Docker and can be executed using Docker Compose. The project is self-contained: it includes the trained model, raw data, generated features, model metrics, a notebook, README documentation, and this written report.

## 2. Data Source

The data source used in this project is the public GitHub REST API. User profiles were collected from GitHub users and organizations discovered through popular repositories and seed queries. The collected raw dataset contains 250 GitHub profiles.

The raw data is stored in:

```
data/raw/github_users.csv
```

The dataset includes fields such as public repositories, public gists, followers, following, account creation date, last update date, hireable status, company, blog, location, account type, and site administrator flag.

A GitHub Personal Access Token can be used through the GITHUB\_TOKEN environment variable to avoid unauthenticated rate limits. The token is not stored in the source code or committed to the repository.

## 3. Churn Definition

For this project, churn is defined as inactivity based on the user's last profile update date. A user is labeled as churned when:

```
days_since_last_activity > 90
```

This means that if a GitHub profile has not shown activity or updates in more than 90 days, it is considered churned for the purpose of this model. The threshold was selected to create a practical activity-based churn definition and a usable class balance for the dataset.

| Class         | Count |
|---------------|-------|
| Active users  | 157   |
| Churned users | 93    |
| Total         | 250   |

## 4. Feature Engineering

The project generates 14 features from the raw GitHub profile data. These features combine activity recency, account maturity, repository behavior, follower/following relationships, profile completeness, and account type.

| Category   | Features                                   |
|------------|--------------------------------------------|
| Time-based | account_age_days, days_since_last_activity |

| Category    | Features                                                                        |
|-------------|---------------------------------------------------------------------------------|
| Ratio       | follower_ratio, followers_per_repo, following_per_repo                          |
| Aggregation | repos_per_year, gists_per_year, engagement_score                                |
| Binary      | has_no_repos, has_company, has_blog, has_location, is_hireable, is_organization |

## 5. Exploratory Data Analysis

The EDA and feature selection notebook is located at:

```
notebooks/eda_and_selection.ipynb
```

The notebook documents raw data loading, feature generation, churn distribution, descriptive statistics, variance threshold analysis, correlation matrix analysis, feature selection comparison, model comparison, and final conclusions.

The variance threshold step identifies features with no or very low variation. The correlation matrix helps detect relationships between features and the churn label, as well as possible redundancy among variables.

## 6. Feature Selection

Four feature selection methods were applied to compare different perspectives on feature importance:

- 1 Filter method using ANOVA F-test
- 2 Recursive Feature Elimination using Logistic Regression
- 3 Decision Tree feature importance
- 4 Random Forest feature importance

Additional filter-based analysis was documented in the notebook through variance threshold and correlation matrix review. The feature selection comparison table is saved in:

```
data/raw/feature_selection_comparison.csv
```

The strongest selected features included:

- days\_since\_last\_activity
- is\_organization
- following\_per\_repo
- is\_hireable

Several other features were classified as optional depending on the selection method, including has\_company, follower\_ratio, engagement\_score, has\_blog, has\_location, and gists\_per\_year.

## 7. Model Training

The final production model uses a Random Forest classifier. The model is trained using the selected operational features and saved to:

```
app/model.pkl
```

The API currently uses the following final features:

- days\_since\_last\_activity
- repos\_per\_year
- follower\_ratio
- followers\_per\_repo
- engagement\_score

The model is exposed through a FastAPI endpoint that receives these features and returns a churn prediction and probability.

## 8. Model Comparison

Because the churn label is directly related to inactivity, a second model variant was trained without the days\_since\_last\_activity feature. This was done to avoid relying only on the same variable used to define churn and to evaluate whether indirect behavioral signals could also predict churn.

The comparison was saved in:

```
data/raw/model_comparison.csv
```

| Model                            | Accuracy | Precision | Recall | F1   |
|----------------------------------|----------|-----------|--------|------|
| With recency                     | 1.00     | 1.00      | 1.00   | 1.00 |
| Without days_since_last_activity | 0.60     | 0.46      | 0.57   | 0.51 |

The model with recency performs perfectly because `days_since_last_activity` is strongly tied to the churn definition. The model without recency has lower but more realistic performance, showing that other signals such as follower behavior, organization status, company information, and engagement can still provide useful predictive value.

## 9. API Endpoints

The FastAPI application exposes three endpoints:

| Endpoint      | Purpose                                                             |
|---------------|---------------------------------------------------------------------|
| GET /health   | Returns the health status of the API.                               |
| GET /features | Returns the list of features expected by the prediction endpoint.   |
| POST /predict | Receives a JSON payload and returns churned plus churn_probability. |

Example prediction payload:

```
{"days_since_last_activity":120,"repos_per_year":1.5,"follower_ratio":0.4,"followers_per_repo":1.0,"engagement_score":30}
```

Example response:

```
{"churned":true,"churn_probability":0.92}
```

## 10. Retention Strategy

Based on the model features, a retention strategy could focus on users with high inactivity and low engagement. Possible actions include:

- Sending reactivation messages to users inactive for more than 90 days.
- Offering personalized recommendations based on repository activity.
- Prioritizing users with previously high engagement scores.
- Monitoring users with declining repository creation or interaction rates.
- Segmenting organizations and individual users separately because their behavior patterns differ.

## 11. Ethical Considerations and Limitations

This project uses public GitHub profile data only. No private repositories, private user information, or sensitive data are collected.

Limitations:

- The dataset is small compared to a production churn system.
- The churn label is inferred from GitHub activity and may not represent actual customer cancellation.
- `days_since_last_activity` is highly predictive because it is directly related to the churn definition.
- GitHub profile update dates may not capture all forms of developer activity.
- The model should be validated with a larger dataset before real-world use.

Despite these limitations, the project demonstrates a complete machine learning workflow including data collection, feature engineering, feature selection, model training, API deployment, and Docker-based execution.

## 12. Deliverables Included

The repository includes all required project deliverables:

- Dockerfile and docker-compose.yml
- FastAPI application with `/health`, `/features`, and `/predict` endpoints
- Trained model: `app/model.pkl`
- Raw and generated datasets in `data/raw/`
- EDA and feature selection notebook: `notebooks/eda_and_selection.ipynb`
- README.md and report.md
- Feature selection and model comparison CSV outputs